

Foundations of R Chapter 9

Robert Gulotty

1/31/2017

This chapter requires many R packages. Take a minute to install each one, then restart R, and continue from there. Install using the command `install.packages("ggmap", type = "source")`

```
library(maptools)
```

```
## Loading required package: sp
```

```
## Checking rgeos availability: TRUE
```

```
library(stringr)
```

```
library(ggplot2)
```

```
library(readr)
```

```
library(dplyr, warn.conflicts = F)
```

```
library(ggmap)
```

```
library(maps)
```

```
library(mapdata)
```

```
library(rgeos)
```

```
## rgeos version: 0.4-2, (SVN revision 581)
```

```
## GEOS runtime version: 3.6.1-CAPI-1.10.1
```

```
## Linking to sp version: 1.3-1
```

```
## Polygon checking: TRUE
```

```
library(sp)
```

```
library(rgdal)
```

```
## rgdal: version: 1.3-6, (SVN revision 773)
```

```
## Geospatial Data Abstraction Library extensions to R successfully loaded
```

```
## Loaded GDAL runtime: GDAL 2.1.3, released 2017/20/01
```

```
## Path to GDAL shared files: /Library/Frameworks/R.framework/Versions/3.5/Resources/library/rgdal/gdal
```

```
## GDAL binary built with GEOS: FALSE
```

```
## Loaded PROJ.4 runtime: Rel. 4.9.3, 15 August 2016, [PJ_VERSION: 493]
```

```
## Path to PROJ.4 shared files: /Library/Frameworks/R.framework/Versions/3.5/Resources/library/rgdal/proj
```

```
## Linking to sp version: 1.3-1
```

Today we will learn about geographic information systems (GIS) and how to interact with them in R.

- The maps package contains the outlines of continents, countries, states, and counties.
- The mapdata package contains a few more, higher-resolution outlines.
- We will use ggplot2 to plot, along with ggmap, which draws on google maps.

We will work with shape files, provided on canvas. To use them, please be sure to download all of the files and keep them in a folder together. You will use R to load in the .shp file, but the others need to be kept in the same directory. Here we use the maptools function, `readShapeSpatial`.

```
shape1 <- readShapeSpatial("~/Dropbox/mathCamp/Code/data/ShapeFile/neweuropemap.shp")
```

```
## Warning: readShapeSpatial is deprecated; use rgdal::readOGR or sf::st_read
```

```
## Warning: readShapePoly is deprecated; use rgdal::readOGR or sf::st_read
```

You can learn a bunch of features of this file using the summary command. There are 34 countries: Albania, Austria, Belgium, Bosnia and Herz., Switzerland, Cyprus, Czech Rep., Germany, Denmark, Spain, Estonia, Finland, France, United Kingdom, Greece, Croatia, Hungary, Ireland, Italy, Kosovo, Lithuania, Luxembourg, Latvia, Macedonia, Malta, Montenegro, Netherlands, Norway, Poland, Portugal, Serbia, Slovakia, Slovenia, Sweden. R deals well with data.frames, but this object is substantially more complicated. The shape file includes `@` is like `$` in that it allows you to pull out elements in an object. We will not cover the details about why we need a more complicated object for maps, but you can read more about it if you search for the S4 system.

Here we are placing a variable called `id` in the data slot of the shape object. Here we are using the `rownames` of the data object. Once we do that, we can use the ggplot function `fortify` to make a data.frame out of the shape file. Recall that ggplot2 operates on data frames. Therefore we need some way to translate the maps data into a data frame format the ggplot can use. Here we are making a variable called `region` from the `id` variable, then we use it to tag each row in the dataframe, creating a column called “id”.

```
shape1@data$id <- rownames(shape1@data)

shape.fort <- fortify(shape1, region = "id")
```

Now, we can recombine this dataframe with the covariates from the shape file.

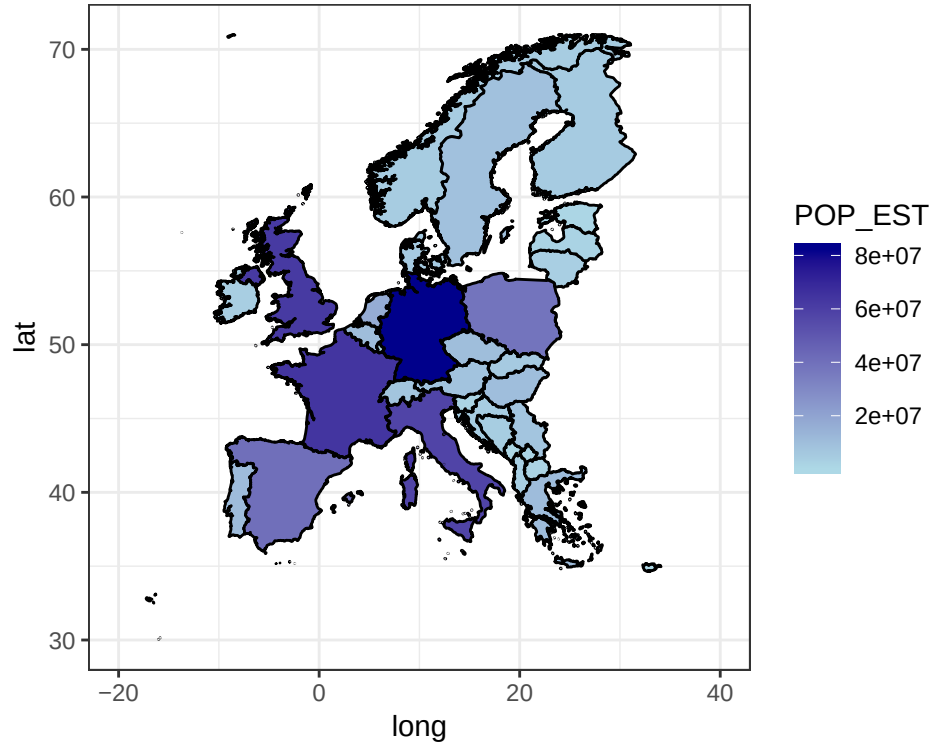
```
shape.data <- shape.fort %>% left_join(data.frame(shape1))
```

```
## Joining, by = "id"
```

In general, it is here that you could add in new data, using the IDs from the file to join in things that might vary by country. We will use the built in population data, but of course you can use data that you collect yourself.

Shape files can be used to plot things as a `geom_polygon`. This takes a lat long, and plots by the group variable. We are also filling these shapes with `POP_EST`.

```
ggplot(data = shape.data, aes(long, lat, group = group)) + geom_polygon(colour = "black",  
  aes(fill = POP_EST)) + scale_fill_gradient(low = "lightblue", high = "darkblue",  
  na.value = "lightyellow") + theme_bw() + xlim(-20, 40) + ylim(30, 71)
```



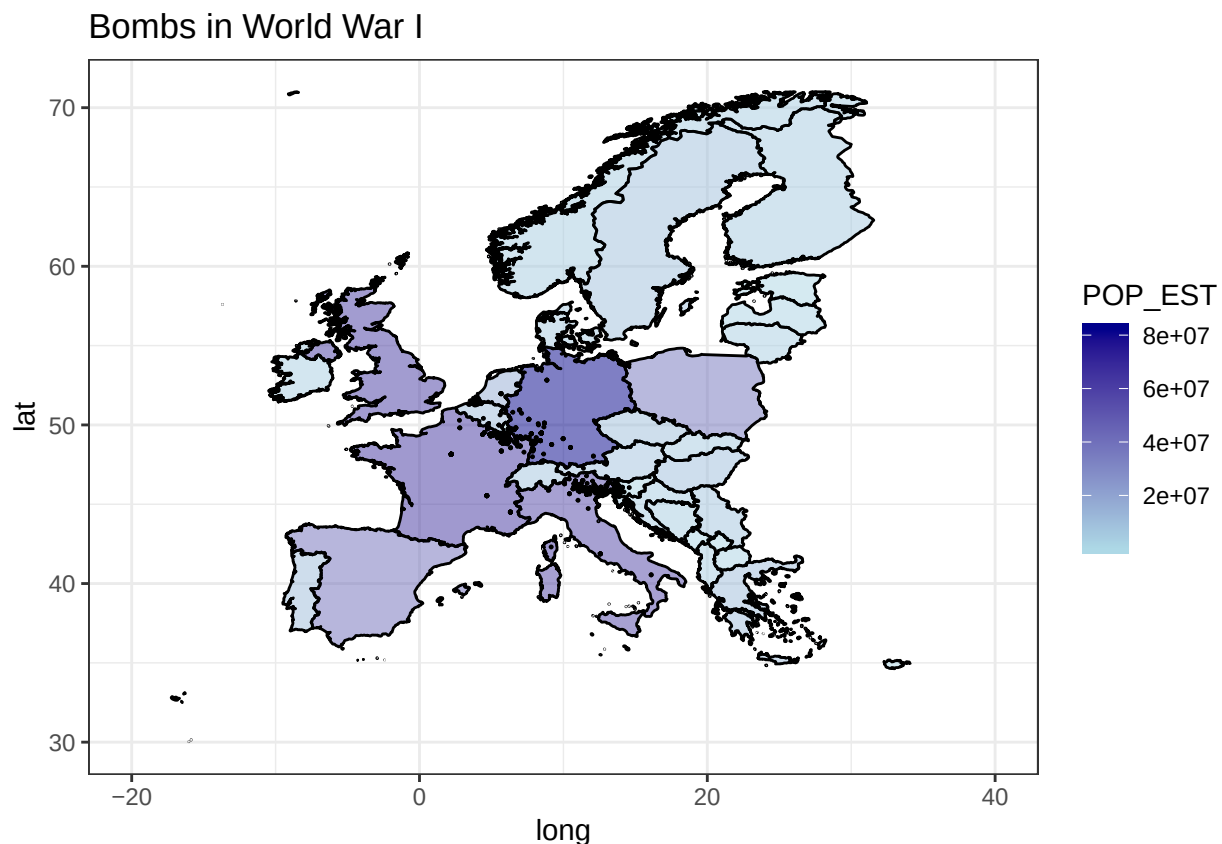
We can do some descriptive analysis using tools from the `sp` package. Here we are loading all of the bombs dropped in Europe during WW1 by the Western Powers.

```
df <- read_csv("~/Dropbox/mathCamp/Code/data/THOR_WW1.csv")

## Parsed with column specification:
## cols(
##   .default = col_character(),
##   id = col_integer()
## )

## See spec(...) for full column specifications.
ggplot() + geom_polygon(data = shape.data, colour = "black", aes(long, lat, group = group,
  fill = POP_EST), alpha = 0.5) + scale_fill_gradient(low = "lightblue", high = "darkblue",
  na.value = "lightyellow") + geom_point(data = df, aes(as.numeric(Lon), as.numeric(Lat)),
  size = 0.2) + xlim(-20, 40) + ylim(30, 71) + theme_bw() + ggtitle("Bombs in World War I")

## Warning in FUN(X[[i]], ...): NAs introduced by coercion
## Warning in FUN(X[[i]], ...): NAs introduced by coercion
## Warning: Removed 81 rows containing missing values (geom_point).
```



Suppose we want to know how many bombs were dropped in France. We can use the `point.in.polygon` command:

```
Francefile <- shape.data[shape.data$NAME == "France", ]

bombsinFrance <- point.in.polygon(point.x = as.numeric(df$Lon), point.y = as.numeric(df$Lat),
```

```
pol.x = Francefile$long, pol.y = Francefile$lat)

## Warning in point.in.polygon(point.x = as.numeric(df$Lon), point.y =
## as.numeric(df$Lat), : NAs introduced by coercion

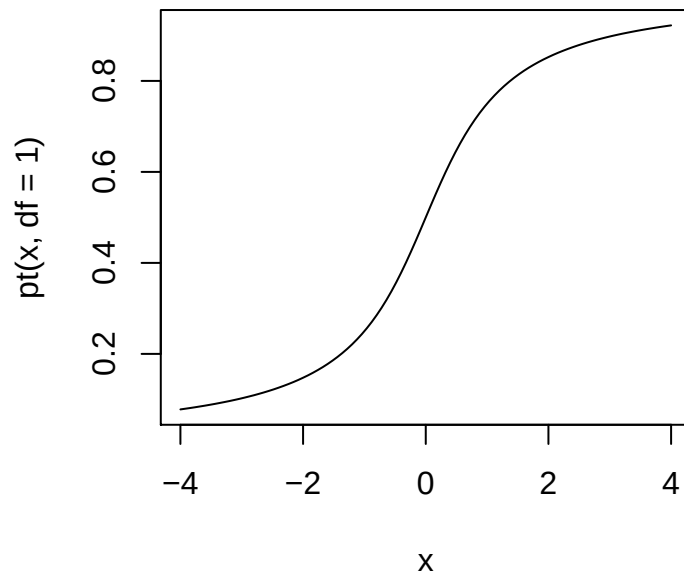
## Warning in point.in.polygon(point.x = as.numeric(df$Lon), point.y =
## as.numeric(df$Lat), : NAs introduced by coercion
table(bombsinFrance)

## bombsinFrance
##    0    1
## 918 523
```

How to simulate a random variable

We can simulate any continuous random variable X with CDF $F_X(x)$ by calculating $F_X^{-1}(U)$ where $U \sim U(0, 1)$. For intuition, consider the following CDF.

```
curve(pt(x, df = 1), from = -4, to = 4)
```



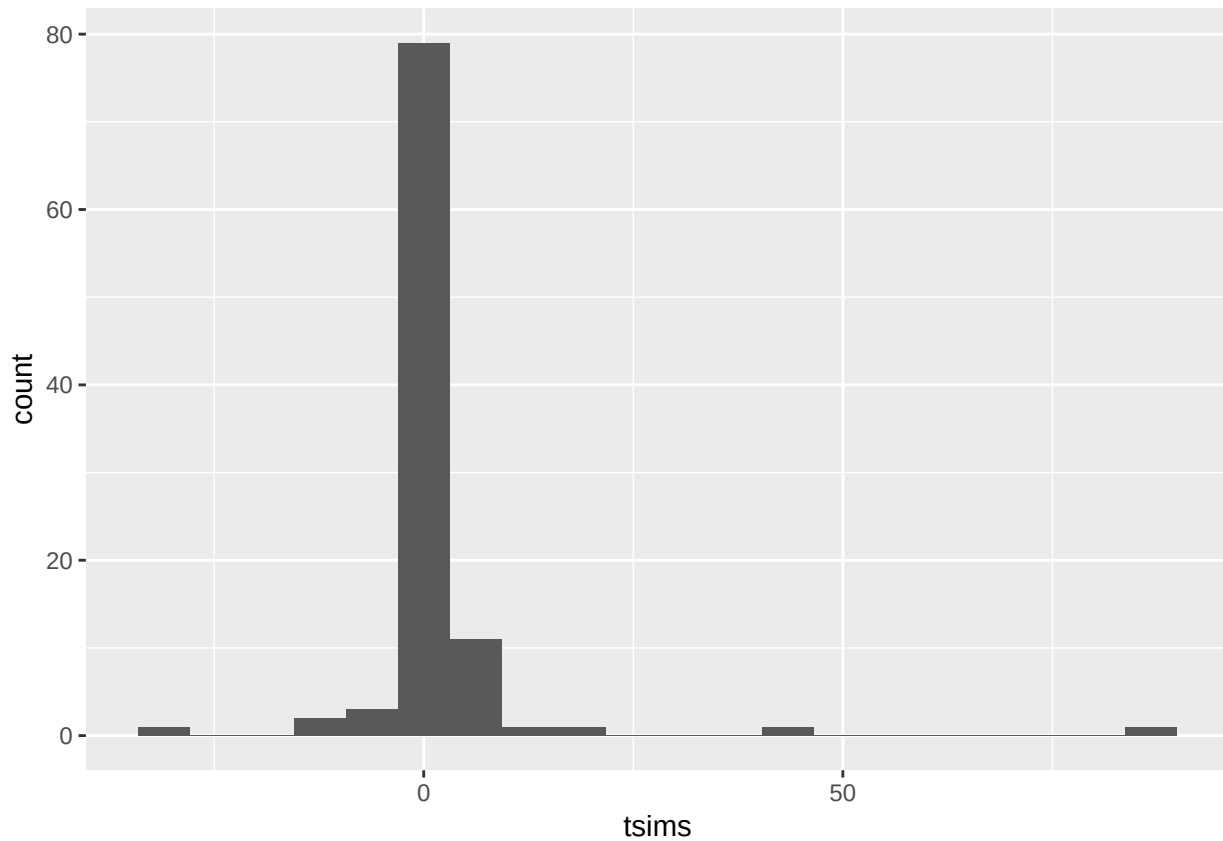
Examining the Y axis, we see numbers from 0 to 1. The mapping from that to the x axis is the inverse of the CDF. Here the equation for the students-t CDF with one degree of freedom is:

$$F(x) = \frac{1}{\pi} \arctan(x) + \frac{1}{2}$$

$$F^{-1}(p) = \tan[\pi(p - 1/2)]$$

This inverse CDF is called the quantile function, and it is built into R with *qt*. So, if we plug in a uniform in for p in the above equation, we will get back a random variable that is distributed t with one degree of freedom. See why the Cauchy distribution has no mean or variance?

```
udraws <- runif(100)
tsims <- tan(pi * (udraws - 1/2))
ggplot() + geom_histogram(aes(tsims), bins = 20)
```



We can examine whether a continuous distribution is equal to a theoretical distribution using a qqplot.

Problems

- 1) Plot the number of bombs dropped in each contemporary European country during World War I.
- 2) Estimate the location (coordinates) of the center of gravity of bombs dropped during World War I using the Thor dataset. Each row of the Thor dataset describes many bombs being dropped. How does that average move over the course of the war? Estimate its location for 1, 3, and 6 month rolling averages.